

第35章 IWDG—独立看门狗

本章参考资料：《STM32F4XX 中文参考手册》IWDG 章节。

学习本章时，配合《STM32F4XX 中文参考手册》IWDG 章节一起阅读，效果会更好，特别是涉及到寄存器说明的部分。

35.1 IWDG 简介

STM32 有两个看门狗，一个是独立看门狗另外一个窗口看门狗，独立看门狗号称宠物狗，窗口看门狗号称警犬，本章我们主要分析独立看门狗的功能框图和它的应用。独立看门狗用通俗一点的话来解释就是一个 12 位的递减计数器，当计数器的值从某个值一直减到 0 的时候，系统就会产生一个复位信号，即 IWDG_RESET。如果在计数没减到 0 之前，刷新了计数器的值的话，那么就不会产生复位信号，这个动作就是我们经常说的喂狗。看门狗功能由 VDD 电压域供电，在停止模式和待机模式下仍能工作。

35.2 IWDG 功能框图剖析

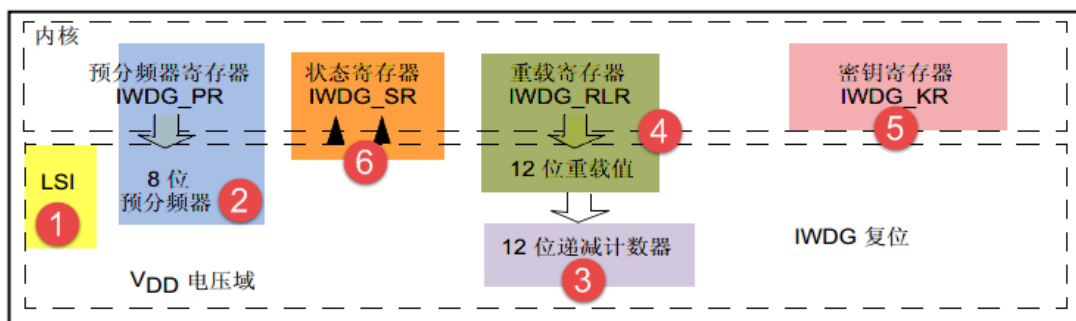


图 35-1 IWDG 功能框图

1. ①独立看门狗时钟

独立看门狗的时钟由独立的 RC 振荡器 LSI 提供，即使主时钟发生故障它仍然有效，非常独立。LSI 的频率一般在 30~60KHZ 之间，根据温度和工作场合会有一定的漂移，我们一般取 40KHZ，所以独立看门狗的定时时间并不一定非常精确，只适用于对时间精度要求比较低的场合。

2. ②计数器时钟

递减计数器的时钟由 LSI 经过一个 8 位的预分频器得到，我们可以操作预分频器寄存器 IWDG_PR 来设置分频因子，分频因子可以是：[4,8,16,32,64,128,256,256]，计数器时钟 $CK_CNT = 40 / 4 * 2^{PRV}$ ，一个计数器时钟计数器就减一。

3. ③计数器

独立看门狗的计数器是一个 12 位的递减计数器，最大值为 0XFFF，当计数器减到 0 时，会产生一个复位信号:IWDG_RESET，让程序重新启动运行，如果在计数器减到 0 之前刷新了计数器的值的话，就不会产生复位信号，重新刷新计数器值的这个动作我们俗称喂狗。

4. ④重装载寄存器

重装载寄存器是一个 12 位的寄存器，里面装着要刷新到计数器的值，这个值的大小决定着独立看门狗的溢出时间。超时时间 $T_{out} = (4 * 2^{prv}) / 40 * rlv (s)$ ，prv 是预分频器寄存器的值，rlv 是重装载寄存器的值。

5. ⑤键寄存器

键寄存器 IWDG_KR 可以说是独立看门狗的一个控制寄存器，主要有三种控制方式，往这个寄存器写入下面三个不同的值有不同的效果。

表格 35-1 键寄存器取值枚举

键值	键值作用
0XAAAA	把 RLR 的值重装载到 CNT
0X5555	PR 和 RLR 这两个寄存器可写
0XCCCC	启动 IWDG

通过写往键寄存器写 0XCCC 来启动看门狗是属于软件启动的方式，一旦独立看门狗启动，它就关不掉，只有复位才能关掉。

6. ⑥状态寄存器

状态寄存器 SR 只有位 0: PVU 和位 1: RVU 有效，这两位只能由硬件操作，软件操作不了。RVU: 看门狗计数器重装载值更新，硬件置 1 表示重装载值的更新正在进行中，更新完毕之后由硬件清 0。PVU: 看门狗预分频值更新，硬件置'1'指示预分频值的更新正在进行中，当更新完成后，由硬件清 0。所以只有当 RVU/PVU 等于 0 的时候才可以更新重装载寄存器/预分频寄存器。

35.3 怎么用 IWDG

独立看门狗一般用来检测 and 解决由程序引起的故障，比如一个程序正常运行的时间是 50ms，在运行完这个段程序之后紧接着进行喂狗，我们设置独立看门狗的定时溢出时间为 60ms，比我们需要监控的程序 50ms 多一点，如果超过 60ms 还没有喂狗，那就说明我们监控的程序出故障了，跑飞了，那么就会产生系统复位，让程序重新运行。

35.4 IWDG 超时实验

35.4.1 硬件设计

- 1、IWDG 一个
- 2、按键一个
- 3、LED 一个

IWDG 属于单片机内部资源，不需要外部电路，需要一个外部的按键和 LED，通过按键来喂狗，喂狗成功 LED 亮，喂狗失败，程序重启，LED 灭一次。

35.4.2 软件设计

我们编写两个 IWDG 驱动文件，bsp_iwdg.h 和 bsp_iwdg.c，用来存放 IWDG 的初始化配置函数。

1. 代码分析

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

IWDG 配置函数

代码清单 35-1 IWDG 配置函数

```
1 /*
2  * 设置 IWDG 的超时时间
3  * Tout = prv/40 * rlv (s)
4  *     prv 可以是[4,8,16,32,64,128,256]
5  * prv:预分频器值，取值如下：
6  *     @arg IWDG_Prescaler_4: IWDG prescaler set to 4
7  *     @arg IWDG_Prescaler_8: IWDG prescaler set to 8
8  *     @arg IWDG_Prescaler_16: IWDG prescaler set to 16
9  *     @arg IWDG_Prescaler_32: IWDG prescaler set to 32
10  *     @arg IWDG_Prescaler_64: IWDG prescaler set to 64
```

```
11 *      @arg IWDG_Prescaler_128: IWDG prescaler set to 128
12 *      @arg IWDG_Prescaler_256: IWDG prescaler set to 256
13 *
14 *      独立看门狗使用 LSI 作为时钟。
15 *      LSI 的频率一般在 30~60KHZ 之间，根据温度和工作场合会有一定的漂移，我
16 *      们一般取 40KHZ，所以独立看门狗的定时时间并不一定非常精确，只适用于对时间精度
17 *      要求比较低的场合。
18 *
19 *      rlv:预分频器值，取值范围为：0-0XFFF
20 *      函数调用举例：
21 *      IWDG_Config(IWDG_Prescaler_64 ,625); // IWDG 1s 超时溢出
22 *          (64/40)*625 = 1s
23 */
24
25 void IWDG_Config(uint8_t prv ,uint16_t rlv)
26 {
27     // 使能 预分频寄存器 PR 和重装载寄存器 RLR 可写
28     IWDG_WriteAccessCmd( IWDG_WriteAccess_Enable );
29
30     // 设置预分频器值
31     IWDG_SetPrescaler( prv );
32
33     // 设置重装载寄存器值
34     IWDG_SetReload( rlv );
35
36     // 把重装载寄存器的值放到计数器中
37     IWDG_ReloadCounter();
38
39     // 使能 IWDG
40     IWDG_Enable();
41 }
```

IWDG 配置函数有两个形参，prv 用来设置预分频的值，具体取值见代码清单 35-2。

代码清单 35-2 形参 prv 取值

```
1 /*
2 *      @arg IWDG_Prescaler_4:      IWDG prescaler set to 4
3 *      @arg IWDG_Prescaler_8:      IWDG prescaler set to 8
4 *      @arg IWDG_Prescaler_16:     IWDG prescaler set to 16
5 *      @arg IWDG_Prescaler_32:     IWDG prescaler set to 32
6 *      @arg IWDG_Prescaler_64:     IWDG prescaler set to 64
7 *      @arg IWDG_Prescaler_128:    IWDG prescaler set to 128
8 *      @arg IWDG_Prescaler_256:    IWDG prescaler set to 256
9 */
```

这些宏在 stm32f4xx_iwdg.h 中定义，宏展开是 8 位的 16 进制数，具体作用是配置配置预分频寄存器 IWDG_PR，获得各种分频系数。形参 rlv 用来设置重装载寄存器 IWDG_RLR 的值，取值范围为 0~0XFFF。溢出时间 $T_{out} = prv/40 * rlv$ (s)，prv 可以是 [4,8,16,32,64,128,256]。如果我们需要设置 1s 的超时溢出，prv 可以取 IWDG_Prescaler_64，rlv 取 625，即调用:IWDG_Config(IWDG_Prescaler_64 ,625)。Tout=64/40*625=1s。

喂狗函数

代码清单 35-3 喂狗函数

```
1 void IWDG_Feed(void)
2 {
3     // 把重装载寄存器的值放到计数器中，喂狗，防止 IWDG 复位
4     // 当计数器的值减到 0 的时候会产生系统复位
5     IWDG_ReloadCounter();
```

6 }

喂狗只是一个俗语，它的实际作用就是在递减计数器的值在没有递减到 0 的时候及时的刷新递减计数器的值，否则将产生系统复位，让程序重新执行。

主函数

代码清单 35-4 主函数

```
1 int main(void)
2 {
3     /* LED 端口初始化 */
4     LED_GPIO_Config();
5
6     Delay(0X8FFFFFFF);
7
8     /* 检查是否为独立看门狗复位 */
9     if (RCC_GetFlagStatus(RCC_FLAG_IWDGRST) != RESET)
10    {
11        /* 独立看门狗复位 */
12        /* 亮红灯 */
13        LED_RED;
14
15        /* 清除标志 */
16        RCC_ClearFlag();
17
18        /*如果一直不喂狗，会一直复位，加上前面的延时，会看到红灯闪烁
19        在 1s 时间内喂狗的话，则会持续亮绿灯*/
20    }
21    else
22    {
23        /*不是独立看门狗复位(可能为上电复位或者手动按键复位之类的) */
24        /* 亮蓝灯 */
25        LED_BLUE;
26    }
27
28    /*初始化按键*/
29    Key_GPIO_Config();
30
31    // IWDG 1s 超时溢出
32    IWDG_Config(IWDG_Prescaler_64 ,625);
33
34
35    //while 部分是在项目中具体需要写的代码，这部分的程序可以用独立看门狗来监控
36    //如果我们知道这部分代码的执行时间，比如是 500ms，那么我们可以设置独立看门狗的
37    //溢出时间是 600ms，比 500ms 多一点，如果要被监控的程序没有跑飞正常执行的话，那么
38    //执行完毕之后就会执行喂狗的程序，如果程序跑飞了那程序就会超时，到达不了喂狗的
39    //程序，此时就会产生系统复位。但是也不排除程序跑飞了又跑回来了，刚好喂狗了，
40    //歪打正着。所以要想更精确的监控程序，可以使用窗口看门狗，窗口看门狗规定必须
41    //在规定的窗口时间内喂狗。
42    while (1)
43    {
44        if ( Key_Scan(KEY1_GPIO_PORT,KEY1_PIN) == KEY_ON )
45        {
46            /* 喂狗，如果不喂狗，系统则会复位，复位后亮红灯，如果在 1s
47            // 时间内准时喂狗的话，则会亮绿灯
48            IWDG_Feed();
49            //喂狗后亮绿灯
50            LED_GREEN;
51        }
52    }
53 }
```

主函数中我们初始化好 LED 和按键相关的配置，设置 IWDG 1s 超时溢出之后，进入 while 死循环，通过按键来喂狗，如果喂狗成功，则亮绿灯，如果喂狗失败的话，系统重启，程序重新执行，当执行到 RCC_GetFlagStatus 函数的时候，则会检测到是 IWDG 复位，然后让红等亮。如果喂狗一直失败的话，则会一直产生系统复位，加上前面延时的效果，则会看到红灯一直闪烁。

我们这里是通过按键来模拟一个喂狗程序，真正的项目中则不是这样使用。while 部分是我们项目中具体需要写的代码，这部分的程序可以用独立看门狗来监控，如果我们知道这部分代码的执行时间，比如是 500ms，那么我们可以设置独立看门狗的溢出时间是 510ms，比 500ms 多一点，如果要被监控的程序没有跑飞正常执行的话，那么执行完毕之后就会执行喂狗的程序，如果程序跑飞了那程序就会超时，到达不了喂狗的程序，此时就会产生系统复位，但是也不排除程序跑飞了又跑回来了，刚好喂狗了，歪打正着。所以要想更精确的监控程序，可以使用窗口看门狗，窗口看门狗规定必须在规定的窗口时间内喂狗，早了不行，晚了也不行。

35.4.3 下载验证

把编译好的程序下载到开发板，在 1s 的时间内通过按键来不断的喂狗，如果喂狗失败，红灯闪烁。如果一直喂狗成功，则绿灯常亮。